

机器学习复习

基本概念

1. 分类、回归的概念区别

分类、回归都是对输入做出预测。就是根据特征，分析输入的内容，判断它的类别，或者预测其值。

分类问题输出的是物体所属的类别，回归问题输出的是物体的值；

分类问题输出的值是离散的，回归问题输出的值是连续的；

分类问题输出的值是定性的，回归问题输出的值是定量的。

2. 训练集、验证集、测试集

训练过程中使用的数据称为训练数据，其中每一个样本称为一个训练样本，训练样本组成的集合称为**训练集**，用于拟合参数；

验证集是用于评估模型的数据样本集合，对拟合得到的模型在验证集上进行预测，用于调整模型超参数；

测试集是指用来对最终模型进行无偏估计的样本集合，是用于评估模型最终性能的数据集，这些数据模型在训练和验证阶段都没有见过。

3. 监督学习、非监督学习

监督学习：对具有概念标记的训练样本进行学习，以尽可能对训练样本集外的数据进行标记预测。（数据有特征 + 标签）

非监督学习：对没有概念标记的训练样本进行学习，以发现训练样本集中的结构性知识（数据之间的关系）。（数据有特征无标签）

4. 回归问题、分类问题

见 1.

5. 欠拟合、过拟合

过拟合：是指建立的模型对训练数据拟合的非常好，但是对测试数据的预测能力非常差，不能反应数据的一般性规律。（训练误差不是越小越好）

欠拟合：是指模型不能在训练集上获得足够低的误差。换句话说，就是模型复杂度低，模型在训练集上就表现很差，没法学习到数据背后的规律。（训练误差太大）

6. 泛化

泛化是指模型很好地拟合以前未见过的数据的能力。

模型适用于新样本的能力，称为泛化能力。

（泛化误差越小越好）

7. 概率与频率的关系

频率是某个事件出现的次数除以总的次数，频率是通过已知的某个具体事件来得出结果；

概率是某一事件所固有的性质。

概率和频率的关系是：通过经验频率我们得到在未来发生某一事件的可能性概率。

所以如果我们能够得到事件发生的频率，那么就可以预估在相同条件下，事件在未来的一次中发生的概率。

8. 独立同分布

假设样本空间中全体样本服从同一个未知“分布”，我们获得的每个样本都是独立地从这个分布上采样获得的。

9. 先验概率、后验概率

先验概率：指根据以往经验和分析，在实验或采样前就可以得到的概率。多项式拟合中，目的是寻找模型参数，在观察数据之前，先假设的概率分布为 $p(w)$ ， $p(w)$ 就是所谓的先验概率。

后验概率：指某件事已经发生，想要计算这件事发生的原因是由某个因素引起的概率。 $p(w|D)$ 被称为后验概率。其意义就是在观察 $D = \{t_1 \dots t_n\}$ 后，对参数向量不确定度的评价。

$$p(w | D) = \frac{p(D | w)p(w)}{p(D)}$$

10. 朴素贝叶斯

朴素贝叶斯分类是以贝叶斯定理为基础并且假设特征条件之间相互独立的分类方法。

设有样本数据集 $D = \{d_1, d_2, \dots, d_n\}$ ，对应样本数据的特征属性集为 $X = \{x_1, x_2, \dots, x_d\}$ ，

类变量为 $Y = \{y_1, y_2, \dots, y_m\}$, 即 D 可以分为 y_m 类别, 其中 $x_1, x_2, \dots, x_d$ 相互独立且随机,

后验概率可以由贝叶斯公式计算出: $P(Y|X) = \frac{P(Y)P(X|Y)}{P(X)}$

朴素贝叶斯基于各特征之间相互独立, 在给定类别为 y 的情况下, 上式可以进一步表示为下式:

$$P(X|Y=y) = \prod_{i=1}^d P(x_i|Y=y)$$

$P(X)$ 的大小固定不变, 因此可以得到一个样本数据属于类别 y_i 的朴素贝叶斯计算:

$$P(y_i|x_1, x_2, \dots, x_d) = \frac{P(y_i) \prod_{j=1}^d P(x_j|y_i)}{\prod_{j=1}^d P(x_j)}$$

(例子: 贝叶斯垃圾邮件分类)

11. 熵

信息量: 理想条件下最短平均编码长度。

$h(x)$ 表示事件发生时获得的信息量, $h(x) = -\log_2 p(x)$ 。

随机变量 x 的熵是关于信息量的期望, 即

$$H[x] = \sum p(x) h(x) = - \sum_x p(x) \log_2 p(x)$$

信息熵通常用来描述整个随机分布所带来的信息量平均值, 代表了随机分布的混乱程度。

12. 连续分布的最大熵

对 $H[x] = -\int p(x) \ln p(x) dx$ 用拉格朗日法求最大熵:

约束条件:

$$\begin{aligned} \int_{-\infty}^{\infty} p(x) dx &= 1 \\ \int_{-\infty}^{\infty} xp(x) dx &= \mu \\ \int_{-\infty}^{\infty} (x - \mu)^2 p(x) dx &= \sigma^2. \end{aligned}$$

使用拉格朗日法：

$$-\int_{-\infty}^{\infty} p(x) \ln p(x) dx + \lambda_1 \left(\int_{-\infty}^{\infty} p(x) dx - 1 \right) \\ + \lambda_2 \left(\int_{-\infty}^{\infty} xp(x) dx - \mu \right) + \lambda_3 \left(\int_{-\infty}^{\infty} (x - \mu)^2 p(x) dx - \sigma^2 \right)$$

问题转变为：在上面的约束条件下，当 $p(x)$ 为什么分布的时候，下面式子最小。可以推导出，对于均值和方差为定值的连续分布，最大熵的概率分布为正态分布。

13. 回归分析法、回归方程

回归分析法：指将具有相关关系的两个变量之间的数量关系进行测定，通过建立一个数学表达式（回归方程）进行统计估计和预测的统计研究方法。

根据因变量和自变量的个数分为：一元回归分析和多元回归分析；

根据因变量和自变量的函数表达式分为：线性回归分析和非线性回归分析。

回归方程是反映自变量和因变量之间联系的数学表达式。

14. 类别不平衡问题

类别不平衡是指分类任务中不同类别的训练样例数目差别很大的情况。会对学习过程造成困扰，如 998 个反例 2 个正例。

15. 信息增益的缺陷

如果属性划分每个分支结点仅包含一个样本，这些分支结点的纯度最大，其信息增益然远大于其他候选属性，但是这样的决策树不具有泛化能力，无法对新样本进行有效预测，信息增益对可取值数目较多的属性有偏好。

基本问题

1. 机器学习的基本过程、三要素

基本过程：

- (1) 数据收集
- (2) 数据预处理：数据清洗、标准化
- (3) 选择模型：根据问题的性质（分类、回归、聚类等）选择合适的机器学习模型
- (4) 训练模型：使用训练集上的数据来训练模型，调整模型参数以最小化损失函数，使用验证集来调整模型参数

(5) 评估模型：使用测试集来评估模型的性能

(6) 模型优化

三要素：

模型：学习什么样（形式）的模型

策略：学习策略就是从假设空间选取最优模型的依据，是用来衡量这个模型对训练集学的好不好的指标

算法：计算出想要的模型呢，求解出最优模型的参数值

2. 最大似然估计

是一种统计估计方法，用于估计模型的参数，使得观测到的样本数据在给定模型下的概率最大化。

假设产生这些数据点的是一个均值为 μ ，方差为 σ^2 的高斯分布，这两个参数的具体值还不知道。

似然函数（likelihood function）如下：

$$p(\mathbf{x}|\mu, \sigma^2) = \prod_{n=1}^N \mathcal{N}(x_n|\mu, \sigma^2)$$

它的直观意义是通过不断调整 μ 、 σ^2 参数，让各个数据点的高斯值相乘，使得 $p(x|\mu, \sigma^2)$ 最大，那么对应的 μ 、 σ^2 就是所要求的高斯分布的参数。

3. 最小二乘法

是一种用于拟合数学模型和估计模型参数的常用优化方法。它的目标是 minimize 观测值与模型预测值之间的残差平方和。

$$(w^*, b^*) = \arg \min_{(w, b)} \sum_{i=1}^m (f(x_i) - y_i)^2 = \arg \min_{(w, b)} \sum_{i=1}^m (y_i - wx_i - b)^2$$

即找到一直线，使所有样本到直线的欧氏距离之和最小。

4. 过拟合的解决办法

正则化：正则化是通过在损失函数中添加一个惩罚项，限制模型参数的大小，从而降低过拟合的风险。常见的正则化包括 L1 正则化和 L2 正则化。

减少模型复杂度：通过减少参数的数量或者选择更简单的模型结构。例如，在神经网络中可以减少隐藏层的节点数或者减少层数。

增加数据量：增加训练数据量是减轻过拟合的有效方法。更多的数据有助于模型更好地学习数据的真实分布，减少对噪声的过拟合。

dropout: 在神经网络中使用 Dropout 层，随机地在训练过程中禁用一些神经元，以减少神经网络的复杂度，有助于防止过拟合。

5. 决策树的基本结构

决策树由节点和分支组成

节点有三种类型：

根节点（包含样本全集）

内部节点（对应于某个属性上的测试）

叶节点（对应一个预测结果）。

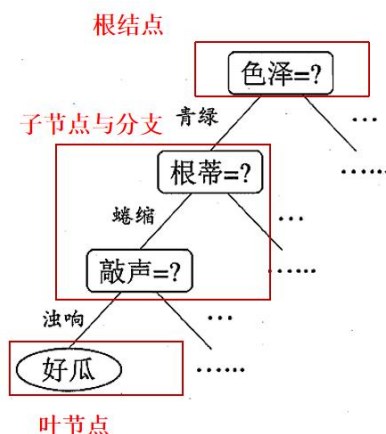


图 4.1 西瓜问题的一棵决策树

一般地，一棵决策树包含一个根节点，若干个内部节点和若干个叶节点

分支：用于连接各个节点，对应于测试的一种可能结果（属性取某个值）。

6. 线性模型的衍生和广义线性模型

对于样例 $(\mathbf{x}, y), y \in R$ ，若希望线性模型的预测值逼近真实标记，则得到线性回归

模型 $y = \mathbf{w}^T \mathbf{x} + b$ 。

广义线性模型一般形式

考虑单调可微函数 $g(\cdot)$

$$y = g^{-1}(\mathbf{w}^T \mathbf{x} + b)$$

$$g(y) = \mathbf{w}^T \mathbf{x} + b$$

g 称为联系函数

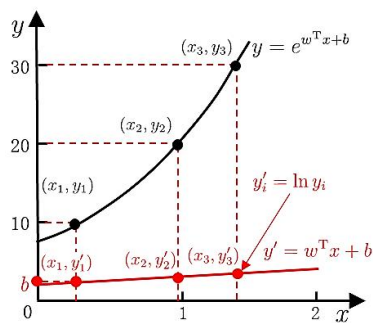
g 不同，就有不同的线性回归

例子：

■ 如果输出的标记是在指数尺度上变化

- 若令 $\ln y = \mathbf{w}^T \mathbf{x} + b$ ，就是对数线性回归

- 实际是在用 $e^{\mathbf{w}^T \mathbf{x} + b}$ 逼近 y
- 实质上在求取输入空间到输出空间的非线性函数映射

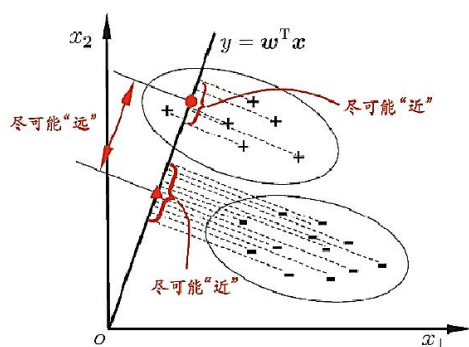


7. LDA（线性判别分析）的思想

给定训练样例集，设法将训练样例投影到一条直线上

使得同类样例投影点尽可能接近、异类样例投影点尽可能远离

将新样本投影到这条直线上，根据投影点位置来确定新样本的类别



8. 多分类学习的思路

思路

有些二分类学习方法直接推广到多分类

通常基于一些基本策略，利用二分类学习器解决多分类问题

拆解法

多分类任务拆为若干个二分类任务求解

- 对问题进行拆分，为拆出的每个二分类任务训练一个分类器
- 测试时，对这些分类器的预测结果进行集成以获得最终的多分类结果
- 关键：如何对多分类任务进行拆分，如何对多个分类器进行集成

9. 拆解法的类型

给定数据集 $D = \{(x_1, y_1), \dots, (x_i, y_i), \dots, (x_m, y_m)\}$, $y_i \in \{C_1, C_2, \dots, C_N\}$

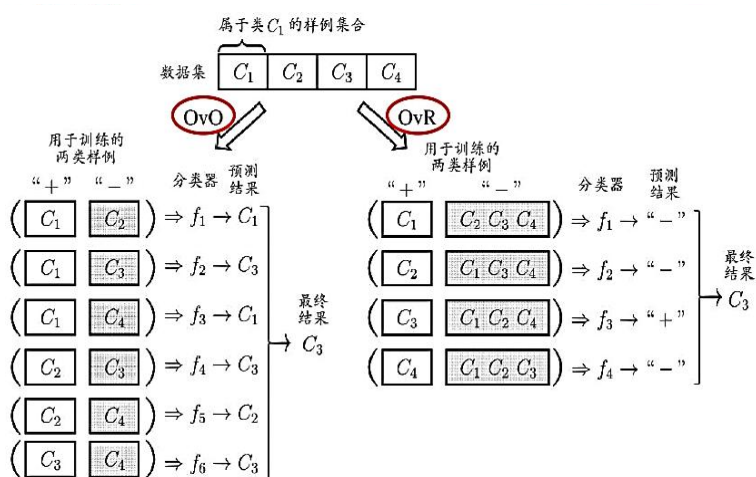
一对一拆分策略(One vs. One, OvO)

- N 个类别两两配对，产生 $N(N-1)/2$ 个二分类任务

- 为类别 C_i 和 C_j 训练一个分类器，把 C_i 类样例作为正例， C_j 类样例作为反例
- 测试阶段，新样本同时提交给所有分类器，得到 $N(N-1)/2$ 个分类结果，把被预测的最多类别作为最终结果

一对其余(One vs.Rest, OvR)

- 每次将一个类的样例作为正例，其他类的样例作为反例，训练 N 个分类器
- 测试时若仅有一个分类器预测为正类，则对应的类别标记作为最终分类结果
- 若有多个分类器预测为正类，则通常考虑各分类器的预测置信度，选择置信度最大的类别标记



多对多(Many vs.Many, MvM)

每次将若干类作为正类，若干其他类作为反类

正反类构造的一种设计：纠错输出码(ECOC) (ppt)

10. 类别不平衡问题的解决思路

■ 假设二分类 $y = w^T x + b$

- 进行分类时，用预测出的 y 值与阈值比较，如 $y > 0.5$ 时为正例，否则为反例
 - y 实际上表达了正例的可能性，几率 $y/(1-y)$ 反映了正例可能性与反例可能性之比
 - 若阈值为 0.5，分类器决策规则可写为 “ $y/(1-y) > 1$ ，则为正例”
- 当训练集正反例的数目不同，令 m^+ 表示正例数， m^- 表示反例数，则观测几率是 m^+/m^- ，因此

$$\text{若 } \frac{y}{1-y} > \frac{m^+}{m^-}, \text{ 则为正例}$$

- 再缩放：对其预测值进行调整 $\frac{y'}{1-y'} = \frac{y}{1-y} \times \frac{m^-}{m^+}$
 - 简单，但实际不易，因为未必能基于训练集观测几率来推断真实几率，训练集不是无偏采样
- 欠采样
 - 去除一些反例使正反例数目接近，然后进行学习
 - 学习时间开销小
- 过采样
 - 增加一些正例使正反例数目接近，然后进行学习
 - 学习时间开销大
 - 不能简单进行重复采样，否则过拟合
 - 对正例进行插值来产生额外的正例
- 阈值移动
 - 基于原始训练集进行学习，将上式嵌入到决策过程中

11. 决策模型的基本流程

见基本算法 4.

12. 信息增益的形式

样本集 D 中第 k 类样本的比例为 p_k ，则 D 的信息熵 $Ent(D) = -\sum_{k=1}^{|y|} p_k \log p_k$ ， $Ent(D)$

的值越小，则 D 的纯度越高。

设离散属性 a 有 V 个可能的取值 $\{a^1, a^2, \dots, a^V\}$ ，

用 a 划分样本集 D ，会产生 V 个分支结点，第 v 个分支结点包含属性 a 取值为 a^v 的样本，记为 D^v ，

不同分支结点所包含的样本数不同，影响不同，给分支结点赋予权重 $|D^v|/|D|$

于是可得属性 a 划分样本集 D 的信息增益

$$\text{Gain}(D, a) = \text{Ent}(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Ent}(D^v)$$

划分前的信息熵
划分后的信息熵

第 v 个分支的权重，
样本越多越重要

13. 剪枝处理的基本策略

剪枝处理的原因：为尽可能正确分类训练样本，结点划分不断重复，会造成决策树分支过多，导致过拟合，可通过去掉一些分支来降低过拟合的风险，剪枝是决策树对付“过拟合”的主要手段。剪枝方法和程度对决策树泛化性能的影响更为显著。

预剪枝：提前终止某些分支的生长

对结点在划分前先进行估计

若当前结点的划分不能带来泛化性能提升，则停止划分并将当前结点标记为叶结点

优点：很多分支都没有展开，降低了过拟合的风险，减少了训练时间开销和测试时间开销。

缺点：有些分支的当前划分虽不能提升泛化性能，但在其基础上的后续划分却有可能显著提高性能，预剪枝基于贪心本质禁止这些分支展开，给预剪枝决策树带来了欠拟合的风险。

后剪枝：生成一棵完全树，再剪枝

先生成完整决策树

然后自底向上对非叶结点进行考察

若将该结点子树替换为叶结点能带来泛化性能提升，则将该子树替换为叶结点

优点：保留了更多的分支，欠拟合风险很小，泛化性能往往优于预剪枝决策树。

缺点：在生成完全决策树之后进行，自底向上对所有非叶结点进行考察，训练开销大。

泛化性能评估：预留部分数据用作验证集进行性能评估。

14. 支持向量机的基本原理

支持向量机（SVM）是一种用于分类和回归的 supervised 学习算法。其基本原理是在特征空间中找到一个决策面，将不同类别的样本分开，并且使得离决策面最近的样本点到决策面的距离最大化。

15. 集成学习主要解决的问题

通过构建并结合多个学习器来完成学习任务，其主要解决以下几个问题：

- **降低过拟合风险：**集成学习通过组合多个模型，降低了单一模型的过拟合风险。减少了对单一模型的依赖，集成的泛化性能通常显著优于单个学习器的泛化性能。
- **提高模型的性能：**集成学习的核心思想是通过组合多个弱学习器，构建一个更强大的模型。即使个别弱学习器可能存在错误，集成学习通过综合各个模型的预测，提高了整体模型的性能。

根据个体学习器的生成方式集成学习方法分为两大类

序列化方法：个体学习器间存在强依赖关系、必须串行生成

并行化方法：个体学习器间不存在强依赖关系、可同时生成

16. 神经网络的激活函数

激活函数是神经网络中一种非线性的函数，它接受神经元的输入并产生输出。激活函数的引入是为了使神经网络能够学习复杂的非线性关系和模式，增加网络的表达能力。

包括：

Sigmoid 函数

Sigmoid 函数将输入映射到范围 $(0, 1)$ 之间，常用于二分类问题中输出概率。

tanh 函数

tanh 函数将输入映射到范围 $(-1, 1)$ 之间，相较于 Sigmoid，它具有零中心性，对于梯度的传递更有利。

ReLU 函数

ReLU 是一种简单的非线性激活函数，对于正值直接输出，负值则输出零。ReLU 的形式简单，训练速度快，能防止梯度消失。

Softmax 函数

Softmax 函数常用于多分类问题，将输入转化为概率分布

17. BP 神经网络的学习过程

BP 是一个迭代学习算法，在迭代的每一轮中采用广义感知机学习规则对参数进行估计 $w \leftarrow w + \Delta w$ 。

过程

- 随机初始化网络中所有的连接权值和阈值
- 前向传播, 计算隐藏层和输出层的输出
- 根据输出结果与训练集计算均方误差
- 误差反向传播, 使用梯度下降法, 以目标的负梯度方向对参数进行调整, 最小化误差函数 E
- 更新权重与偏置

输入: 训练集 $D = \{(x_k, y_k)\}_{k=1}^m$;
学习率 η .

过程:

- 1: 在 $(0, 1)$ 范围内随机初始化网络中所有连接权和阈值
- 2: **repeat**
- 3: **for all** $(x_k, y_k) \in D$ **do**
- 4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{y}_k$;
- 5: 根据式(5.10) 计算输出层神经元的梯度项 g_j ;
- 6: 根据式(5.15) 计算隐层神经元的梯度项 e_h ;
- 7: 根据式(5.11)-(5.14) 更新连接权 w_{hj} , v_{ih} 与阈值 θ_j , γ_h
- 8: **end for**
- 9: **until** 达到停止条件

输出: 连接权与阈值确定的多层前馈神经网络

将示例输入给输入层神经元, 逐层将信号前传, 直到产生输出层结果, 然后计算输出层误差

将误差逆向传播到隐层神经元

根据隐层神经元误差对连接权和阈值进行调整

训练误差达到一个很小的值, 与缓解过拟合的策略有关

图 5.8 误差逆传播算法

基本算法

1. 一元线性回归的基本形式和参数求解

给定一维属性数据集 $D = \{(x_i, y_i)\}_{i=1}^m$, 要得到 $f(x_i) = wx_i + b$, 使 $f(x_i) \approx y_i$ 。

确定 w 、 b 。

令均方误差最小化:

$$(w^*, b^*) = \arg \min_{(w, b)} \sum_{i=1}^m (f(x_i) - y_i)^2 = \arg \min_{(w, b)} \sum_{i=1}^m (y_i - wx_i - b)^2$$

令 $E_{(w, b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$, 分别对 w 、 b 求导并令导数为 0, 可得

$$w = \frac{\sum_{i=1}^m y_i (x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m} \left(\sum_{i=1}^m x_i \right)^2} \quad b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)$$

2. 多元线性回归的基本形式和参数求解

多元线性回归

- 对于d维属性数据集

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$$

$$\mathbf{x}_i = (x_{i1}; \dots; x_{id})$$

- 要得到 $f(\mathbf{x}_i) = \mathbf{w}\mathbf{x}_i + b$ ，使 $f(\mathbf{x}_i) \approx y_i$
- 令 $\hat{\mathbf{w}} = (\mathbf{w}; b)$ ， $\mathbf{y} = (y_1; \dots; y_m)$ ，数据集表示为

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1d} & 1 \\ x_{21} & x_{22} & \dots & x_{2d} & 1 \\ \dots & \dots & \dots & \dots & \dots \\ x_{m1} & x_{m2} & \dots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \dots & \dots \\ \mathbf{x}_m^T & 1 \end{pmatrix}$$

多元线性回归

- 采用最小二乘法求解，有

$$\frac{\partial E_{\hat{\mathbf{w}}}}{\partial \hat{\mathbf{w}}} = 2\mathbf{X}^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})$$

- 若 $\mathbf{X}^T \mathbf{X}$ 满秩或正定，则

$$\hat{\mathbf{w}}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- 若 $\mathbf{X}^T \mathbf{X}$ 不满秩，则可解出多个 $\hat{\mathbf{w}}$
 - 此时需求助于归纳偏好，或引入正则化项

- 令 $\hat{\mathbf{x}}_i = (\mathbf{x}_i; 1)$ ，则学得多元线性回归模型为

$$f(\hat{\mathbf{x}}_i) = \hat{\mathbf{x}}_i^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

3. 求解极大似然函数估计的一般步骤

似然函数如下：

$$p(\mathbf{x}|\mu, \sigma^2) = \prod_{n=1}^N \mathcal{N}(x_n|\mu, \sigma^2) \quad f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (\mu \in \mathbb{R}, \sigma > 0)$$

为了计算和分析的方便，可以将似然函数转换成（自然）对数形式：

$$\ln p(\mathbf{x}|\mu, \sigma^2) = -\frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 - \frac{N}{2} \ln \sigma^2 - \frac{N}{2} \ln(2\pi).$$

对对数似然函数关于参数进行求导，并令导数等于零，求解得到参数的最大值求导，得方程组：

$$\begin{cases} \frac{\partial \ln L(\mu, \sigma^2)}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0 \\ \frac{\partial \ln L(\mu, \sigma^2)}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^n (x_i - \mu)^2 = 0 \end{cases}$$

联合解得找到最优的 μ 、 σ^2 ：

$$\mu_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n$$
$$\sigma_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \mu_{\text{ML}})^2$$

求最大似然估计量的一般步骤：

- (1) 写出似然函数；
- (2) 对似然函数取对数，并整理；
- (3) 求导数；
- (4) 解似然方程。

4. 描述决策树的算法流程

策略：分而治之

- (1) 开始构建根节点，选择一个最优特征，分割成子集
- (2) 自根至叶的递归过程，每个中间结点寻找一个“划分”属性
- (3) 所有子集按内部节点的属性递归的进行分割，能够被基本正确分类，构建叶节点
- (4) 每个子集都被分到叶节点上，即都有了明确的类，生成树

三种递归停止条件：

当前结点包含的样本全属于同一类别，无需划分；

当前属性集为空或是所有样本在所有属性上取值相同，无法划分

当前结点包含的样本集合为空，不能划分。

选择最优划分属性：选择能够获得最大信息增益（率）的属性划分。

5. 支持向量机的目标函数推导步骤（见 ppt）

样本点 x 到决策面的距离表示为：

$$\text{distance}(\mathbf{x}, \mathbf{b}, \mathbf{w}) = \left| \frac{\mathbf{w}^T}{\|\mathbf{w}\|} (\mathbf{x} - \mathbf{x}') \right| \stackrel{①}{=} \frac{1}{\|\mathbf{w}\|} |\mathbf{w}^T \mathbf{x} + \mathbf{b}|$$

✓ 数据标签定义

✎ 数据集：(X1,Y1)(X2,Y2)... (Xn,Yn)

✎ Y为样本的类别：当X为正例时候 Y = +1 当X为负例时候 Y = -1

✎ 决策方程： $y(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}) + b$ （其中 $\Phi(\mathbf{x})$ 是对数据做了变换，后面继续说）

$$\begin{aligned} y(x_i) > 0 &\Leftrightarrow y_i = +1 \\ y(x_i) < 0 &\Leftrightarrow y_i = -1 \end{aligned} \Rightarrow y_i \cdot y(x_i) > 0$$

✓ 优化的目标

✎ 通俗解释：找到一个条线（w和b），使得离该线最近的点（雷区）能够最远

✎ 将点到直线的距离化简得： $\frac{y_i \cdot (\mathbf{w}^T \cdot \Phi(x_i) + b)}{\|\mathbf{w}\|}$

（由于 $y_i \cdot y(x_i) > 0$ 所以将绝对值展开原始依旧成立）

✓ 目标函数

✎ 放缩变换：对于决策方程（w,b）可以通过放缩使得其结果值 $|Y| >= 1$

$$\Rightarrow y_i \cdot (\mathbf{w}^T \cdot \Phi(x_i) + b) \geq 1 \quad (\text{之前我们认为恒大于0, 现在严格了些})$$

✎ 优化目标： $\arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \min_i [y_i \cdot (\mathbf{w}^T \cdot \Phi(x_i) + b)] \right\}$

由于 $y_i \cdot (\mathbf{w}^T \cdot \Phi(x_i) + b) \geq 1$ ，只需要考虑 $\arg \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$ （目标函数搞定！）

✓ 目标函数

✎ 当前目标： $\max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$ ，约束条件： $y_i (\mathbf{w}^T \cdot \Phi(x_i) + b) \geq 1$

✎ 常规套路：将求解极大值问题转换成极小值问题 $\Rightarrow \min_{\mathbf{w}, b} \frac{1}{2} \mathbf{w}^2$

✎ 如何求解：应用拉格朗日乘子法求解

✓ 拉格朗日乘子法

✎ 带约束的优化问题： $\min_x f_0(x)$

$$\text{subject to } f_i(x) \leq 0, i=1, \dots, m \\ h_i(x) = 0, i=1, \dots, q$$

✎ 原式转换： $\min L(x, \lambda, v) = f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^q v_i h_i(x)$

✎ 我们的式子： $L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w^T \cdot \Phi(x_i) + b) - 1)$

(约束条件不要忘了： $y_i (w^T \cdot \Phi(x_i) + b) \geq 1$)

✓ SVM求解

✎ 分别对w和b求偏导,分别得到两个条件(由于对偶性质)

$$\min_{w, b} \max_{\alpha} L(w, b, \alpha) \rightarrow \max_{\alpha} \min_{w, b} L(w, b, \alpha)$$

✎ 对w求偏导： $\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i \Phi(x_i)$

对b求偏导： $\frac{\partial L}{\partial b} = 0 \Rightarrow 0 = \sum_{i=1}^n \alpha_i y_i$

✓ SVM求解

✎ 带入原始： $L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w^T \Phi(x_i) + b) - 1)$

$$\text{其中 } w = \sum_{i=1}^n \alpha_i y_i \Phi(x_i) \quad 0 = \sum_{i=1}^n \alpha_i y_i$$

$$= \frac{1}{2} w^T w - w^T \sum_{i=1}^n \alpha_i y_i \Phi(x_i) - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \Phi(x_i) \right)^T \sum_{i=1}^n \alpha_i y_i \Phi(x_i) \quad \rightarrow \text{完成了第一步求解 } \min_{w, b} L(w, b, \alpha)$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \Phi^T(x_i) \Phi(x_j)$$

✓ SVM求解

✎ 继续对α求极大值： $\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\Phi(x_i) \cdot \Phi(x_j))$

$$\text{条件: } \sum_{i=1}^n \alpha_i y_i = 0$$

$$\alpha_i \geq 0$$

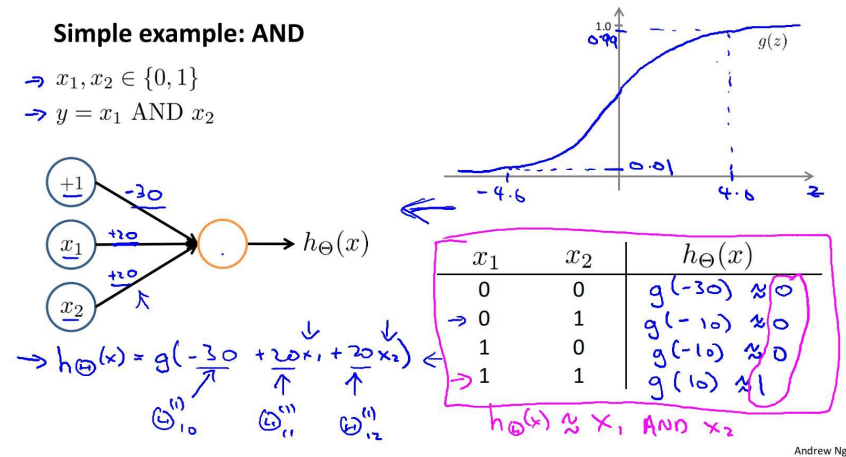
✎ 极大值转换成求极小值： $\min_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\Phi(x_i) \cdot \Phi(x_j)) - \sum_{i=1}^n \alpha_i$

$$\text{条件: } \sum_{i=1}^n \alpha_i y_i = 0$$

$$\alpha_i \geq 0$$

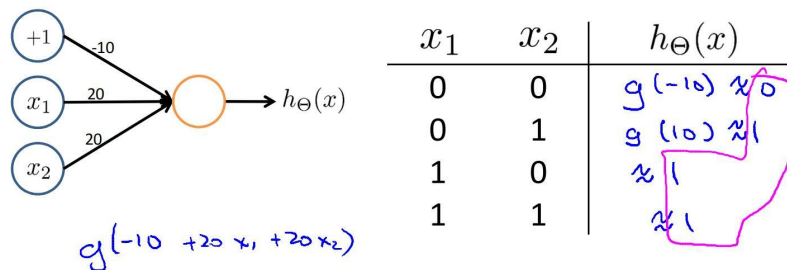
6. 两层神经网络解决异或问题

AND 实现: $a \& b$



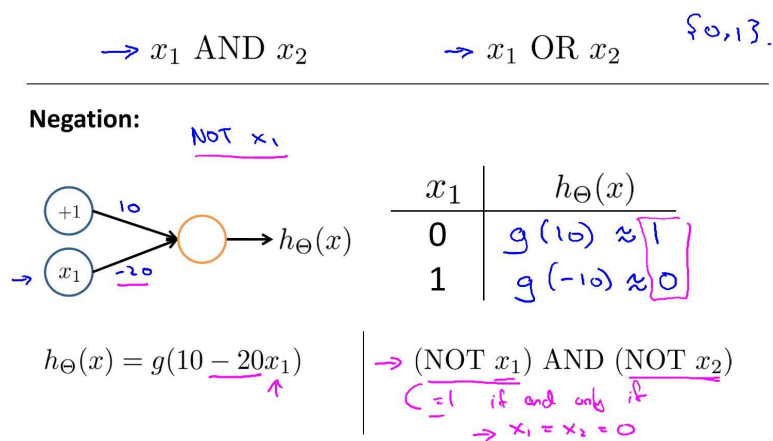
OR 实现: $a | b$

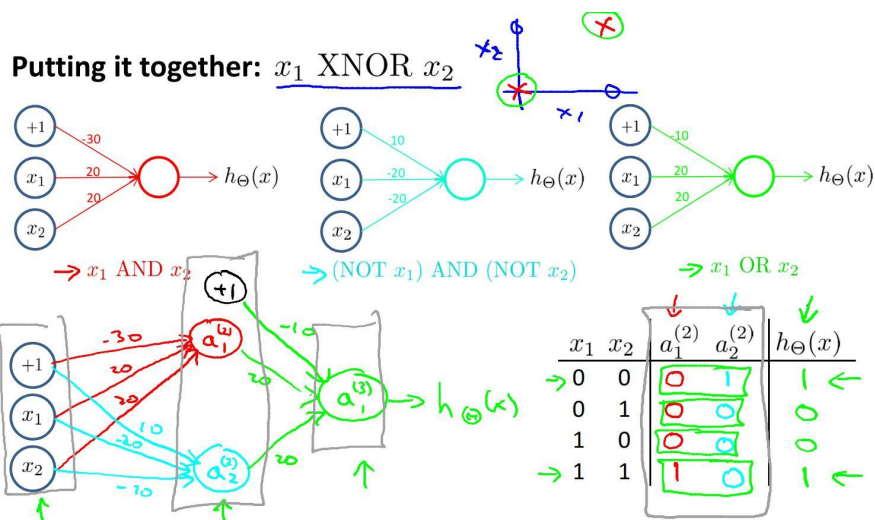
Example: OR function



Andrew Ng

NOT 实现: $\neg a$

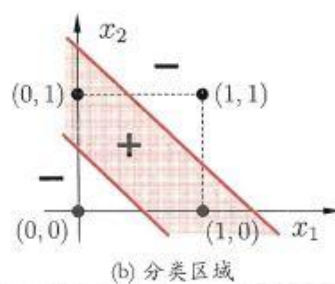
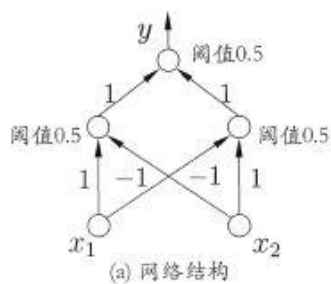




Andrew Ng

XOR 实现: $a \wedge b = (a \& -b) \mid (-a \& b)$

XNOR 实现: $a \text{ XNOR } b = (a \& b) \mid (-a \& -b)$



<https://blog.csdn.net/u012372050>

7. 反向传播算法

见基本问题 17.

8. Bagging 算法过程

思想:

欲得到泛化性能强的集成, 个体学习器应尽可能相互独立

独立在现实中无法做到, 可以设法使基学习器尽可能具有较大的差异

一种可能的方法是对训练样本进行采样, 产生若干不同子集, 再从每个数据子集训练出基学习器。

算法:

- 基于自助采样法

给定包含 m 个样本的数据集, 先随机取出一个样本放入采样集中, 再将其放回初

始数据集，经过 m 次随机采样，得到 m 个样本的采样集，初始训练集中有样本在采样集中多次出现，有的则从未出现

- 采样出 T 个训练样本集，训练出 T 个基学习器，将基学习器进行结合
- 在对预测输出进行结合时，常用简单投票法。对回归任务使用简单平均法

若出现两个类收到同样票数的情形，最简单的做法是随机选择一个，或考察学习器投票的置信度来确定最终胜者。

优势

与训练一个学习器的复杂度同阶；

初始训练集会剩下约 36.8% 样本，可做验证集，可辅助剪枝

从偏差方差看，主要降低方差，对不剪枝决策树、神经网络等易受样本扰动的学习器效果明显